

---

# Product Catalog

Project Report – Automated Software Testing  
Academic Year 2021-2022

---

GitHub Repository:

<https://github.com/mohamedjama-farah/product-catalog>

|                     |                                                        |
|---------------------|--------------------------------------------------------|
| tablerow Student    | Mohamed Fara Jama                                      |
| white Student ID    | 7130441                                                |
| tablerow Course     | Advanced Techniques and Tools for Software Development |
| white Course ID     | B027540 (B059)                                         |
| tablerow Rating     | Low-rating                                             |
| white Java Version  | Java 17 (LTS)                                          |
| tablerow Build Tool | Maven                                                  |
| white Database      | MongoDB                                                |

This report describes the development of a Java Swing desktop application for managing a product catalog, applying all techniques taught in the course: Test-Driven Development, build automation with Maven, and continuous integration with GitHub Actions.

# Contents

---

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction and Project Description</b>        | <b>2</b>  |
| 1.1      | Domain Model . . . . .                             | 2         |
| 1.2      | Application Features . . . . .                     | 2         |
| <b>2</b> | <b>Architecture and Design</b>                     | <b>3</b>  |
| 2.1      | Project Structure . . . . .                        | 3         |
| 2.2      | Component Responsibilities . . . . .               | 4         |
| 2.3      | Design Choices . . . . .                           | 4         |
| <b>3</b> | <b>Applied Techniques and Frameworks</b>           | <b>5</b>  |
| 3.1      | Test-Driven Development (TDD) . . . . .            | 5         |
| 3.2      | Testing Pyramid . . . . .                          | 5         |
| 3.3      | Mocking Strategy . . . . .                         | 5         |
| 3.4      | Code Coverage . . . . .                            | 5         |
| 3.5      | Mutation Testing with PIT . . . . .                | 6         |
| 3.6      | Code Quality with SonarCloud . . . . .             | 6         |
| <b>4</b> | <b>Build Automation and Continuous Integration</b> | <b>7</b>  |
| 4.1      | Maven Build . . . . .                              | 7         |
| 4.2      | GitHub Actions CI . . . . .                        | 7         |
| 4.3      | Docker and Testcontainers . . . . .                | 7         |
| <b>5</b> | <b>Development of the Most Interesting Parts</b>   | <b>8</b>  |
| 5.1      | Controller Testing with Mockito InOrder . . . . .  | 8         |
| 5.2      | Swing GUI Testing with AssertJ-Swing . . . . .     | 8         |
| 5.3      | Integration Tests with Testcontainers . . . . .    | 9         |
| <b>6</b> | <b>Problems Encountered and Solutions</b>          | <b>10</b> |
| <b>7</b> | <b>Build and Test Instructions</b>                 | <b>11</b> |
| 7.1      | Clone the Repository . . . . .                     | 11        |
| 7.2      | Run the Full Build and All Tests . . . . .         | 11        |
| 7.3      | Run Mutation Testing . . . . .                     | 11        |
| 7.4      | Import into Eclipse . . . . .                      | 11        |
| 7.5      | Run Tests from Eclipse . . . . .                   | 11        |
| <b>8</b> | <b>Summary</b>                                     | <b>12</b> |

# 1. Introduction and Project Description

---

The Product Catalog application is a simple Java Swing desktop application that allows users to manage a catalog of products and categories. The application was developed following all the methodologies and tools taught during the course, including Test-Driven Development (TDD), build automation with Maven, and continuous integration with GitHub Actions.

The project is rated as **low-rating**, implementing a straightforward CRUD application with a clean domain model consisting of two entities: **Product** and **Category**, connected by a many-to-one relationship.

## 1.1 Domain Model

The application manages two entities:

- **Product**: has an `id` (String), `name` (String), `price` (double), and `categoryId` (String)
- **Category**: has an `id` (String) and `name` (String)
- **Relation**: Many Products belong to one Category (many-to-one)

bluemid

|                |                             |                     |
|----------------|-----------------------------|---------------------|
| tablerow       | id, name, price, categoryId | Many Products → One |
| Product        |                             | Category            |
| white Category | id, name                    | —                   |

## 1.2 Application Features

The Swing GUI allows the user to:

- View all products in a scrollable list
- Add a new product by entering id, name, price, and category id
- Delete a selected product from the list
- See error messages when operations fail (e.g. product already exists)

## 2. Architecture and Design

---

The application follows the **Model-View-Controller (MVC)** pattern as taught during the course. This separation of concerns makes each component independently testable and maintainable.

### 2.1 Project Structure

```
src/main/java/com/example/productcatalog/  
  app/  
    ProductCatalogApp.java          <- Entry point (excluded  
      from coverage)  
  controller/  
    ProductCatalogController.java    <- Business logic  
  model/  
    Product.java                    <- Domain entity  
    Category.java                   <- Domain entity  
  repository/  
    ProductRepository.java           <- Interface  
    CategoryRepository.java          <- Interface  
  mongo/  
    ProductMongoRepository.java      <- MongoDB implementation  
    CategoryMongoRepository.java     <- MongoDB implementation  
  view/  
    ProductCatalogView.java          <- Interface  
  swing/  
    ProductCatalogSwingView.java     <- Swing implementation  
  
src/test/java/com/example/productcatalog/  
  controller/  
    ProductCatalogControllerTest.java <- Unit tests (Mockito)  
  model/  
    ProductTest.java                 <- Unit tests  
    CategoryTest.java                <- Unit tests  
  repository/mongo/  
    ProductMongoRepositoryTest.java  <- Integration tests  
    CategoryMongoRepositoryTest.java <- Integration tests  
  view/swing/  
    ProductCatalogSwingViewTest.java <- UI unit tests (AssertJ-  
      Swing)  
  e2e/  
    ProductCatalogE2ETest.java       <- End-to-end tests
```

## 2.2 Component Responsibilities

bluedark

|                                            |                                                                      |
|--------------------------------------------|----------------------------------------------------------------------|
| tablerow ProductCatalogController          | Coordinates between view and repository; contains all business logic |
| white<br>ProductCatalogView<br>(interface) | Defines the contract between controller and UI                       |
| tablerow ProductCatalogSwingView           | Implements the Swing GUI; delegates all actions to the controller    |
| white<br>ProductRepository<br>(interface)  | Defines CRUD operations for products                                 |
| tablerow ProductMongoRepository            | Implements the repository using the MongoDB driver                   |
| white<br>ProductCatalogApp                 | Entry point; wires all components together                           |

## 2.3 Design Choices

The **Repository Pattern** was used to abstract the data access layer. Both `ProductRepository` and `CategoryRepository` are defined as interfaces, allowing the controller to depend on abstractions rather than concrete implementations. This makes the controller fully testable with mock objects without requiring a real database.

The **View interface** (`ProductCatalogView`) allows the controller to be tested independently of the Swing framework using Mockito mocks. The Swing implementation is injected at runtime in `ProductCatalogApp`.

## 3. Applied Techniques and Frameworks

---

### 3.1 Test-Driven Development (TDD)

All production code was written following the **Red-Green-Refactor** cycle: a failing test was written first, then the minimum production code to make it pass, followed by refactoring without breaking tests. This discipline ensured that every line of production code is covered by at least one test.

### 3.2 Testing Pyramid

The project respects the shape of the testing pyramid:

bluedark

|                            |                |                             |                          |
|----------------------------|----------------|-----------------------------|--------------------------|
| tablerow Unit<br>(model)   | 20             | JUnit 4 + AssertJ           | Test POJOs               |
| white Unit<br>(controller) | 6              | JUnit 4 + Mockito           | Test business logic      |
| tablerow Unit<br>(view)    | 12             | AssertJ-Swing +<br>Mockito  | Test Swing<br>components |
| white Integration          | 11             | JUnit 4 +<br>Testcontainers | Test MongoDB repos       |
| tablerow<br>End-to-End     | 2              | AssertJ-Swing + TC          | Full application test    |
| white <b>Total</b>         | <b>51 + 1</b>  |                             |                          |
|                            | <b>skipped</b> |                             |                          |

### 3.3 Mocking Strategy

Mockito was used to mock **only project-owned types**: `ProductRepository`, `CategoryRepository`, and `ProductCatalogView` interfaces. Third-party types such as `MongoClient` were **never mocked**. Instead, `Testcontainers` was used to start a real MongoDB instance during integration and end-to-end tests, as required by the exam guidelines.

### 3.4 Code Coverage

100% code coverage was achieved on both **Coveralls** and **SonarCloud**. The only class excluded from coverage is `ProductCatalogApp`, which is the entry point that wires all components together. This exclusion is justified as taught in the course, since the entry point cannot be meaningfully unit tested in isolation.

### 3.5 Mutation Testing with PIT

PIT mutation testing was configured to target only the controller and model packages. This avoids excessive build times while ensuring the most critical code is thoroughly tested.

```
>> Generated 8 mutations Killed 8 (100%)  
>> BUILD SUCCESS -- 0 surviving mutants
```

### 3.6 Code Quality with SonarCloud

bluedark

|                          |               |
|--------------------------|---------------|
| tablerow Security        | A — 0 issues  |
| white Reliability        | A — 0 issues  |
| tablerow Maintainability | A — 0 issues  |
| white Coverage           | 100%          |
| tablerow Duplications    | 0%            |
| white Quality Gate       | <b>PASSED</b> |

## 4. Build Automation and Continuous Integration

---

### 4.1 Maven Build

Maven is used as the build automation tool. The POM configures all necessary plugins:

- `maven-compiler-plugin`: compiles Java 17 source code
- `maven-surefire-plugin`: runs unit tests with Java 17 module opens
- `maven-failsafe-plugin`: runs integration and end-to-end tests
- `jacoco-maven-plugin`: generates code coverage reports (XML + HTML)
- `pitest-maven`: mutation testing on controller and model only
- `coveralls-maven-plugin`: sends coverage reports to Coveralls
- `sonar-maven-plugin`: sends analysis results to SonarCloud

### 4.2 GitHub Actions CI

A GitHub Actions workflow runs on every push to the `main` branch on **Ubuntu Linux** (`ubuntu-latest`):

- Checkout with `fetch-depth: 0` (required for SonarCloud)
- Set up Java 17 Eclipse Temurin distribution
- Cache Maven dependencies for faster builds
- Run `xvfb-run mvn clean verify` for headless GUI testing
- Send coverage reports to Coveralls
- Run SonarCloud analysis

### 4.3 Docker and Testcontainers

Docker is used exclusively through **Testcontainers**, which automatically starts a MongoDB container before tests and stops it afterward. No containers are started manually from GitHub Actions steps, as required by the exam guidelines. MongoDB 6.0 is used for integration tests and MongoDB 4.4.3 for end-to-end tests.



## 5. Development of the Most Interesting Parts

---

### 5.1 Controller Testing with Mockito InOrder

The most interesting aspect of the controller tests is the use of **InOrder** verification to ensure the correct sequence of interactions between the controller and its dependencies:

```
@Test
public void testNewProductWhenProductDoesNotExist() {
    Product product = new Product("1", "Tablet", 299.99, "cat1");
    when(productRepository.findById("1")).thenReturn(null);

    productCatalogController.newProduct(product);

    InOrder inOrder = inOrder(productRepository,
        productCatalogView);
    inOrder.verify(productRepository).save(product);
    inOrder.verify(productCatalogView).productAdded(product);
}

@Test
public void testNewProductWhenProductAlreadyExists() {
    Product existing = new Product("1", "Laptop", 999.99, "cat1");
    ;
    when(productRepository.findById("1")).thenReturn(existing);

    productCatalogController.newProduct(existing);

    verify(productCatalogView)
        .showError("Already existing product with id 1", existing
    );
    verifyNoMoreInteractions(ignoreStubs(productRepository));
}
```

### 5.2 Swing GUI Testing with AssertJ-Swing

The Swing view was tested using AssertJ-Swing, which provides a fluent API for interacting with GUI components in tests. Tests run on the Event Dispatch Thread using `GuiActionRunner.execute()`, ensuring thread safety.

```
@Test
public void
    testWhenAllFieldsAreNonEmptyThenAddButtonShouldBeEnabled() {
    window.textBox("idTextBox").enterText("1");
    window.textBox("nameTextBox").enterText("Laptop");
    window.textBox("priceTextBox").enterText("999.99");
```

```
        window.textBox("categoryIdTextBox").enterText("cat1");
        assertThat(addButton().isEnabled()).isTrue();
    }
```

### 5.3 Integration Tests with Testcontainers

Integration tests start a real MongoDB instance automatically. The `MongoClient` is created using the container's connection string, ensuring no third-party mocking:

```
@Before
public void setUp() {
    mongoDBContainer = new MongoDBContainer("mongo:6.0");
    mongoDBContainer.start();
    mongoClient = MongoClient.create(mongoDBContainer.
        getReplicaSetUrl());
    repository = new ProductMongoRepository(mongoClient, "testdb"
    );
}

@After
public void tearDown() {
    if (mongoClient != null) mongoClient.close();
    if (mongoDBContainer != null) mongoDBContainer.stop();
}
```

## 6. Problems Encountered and Solutions

---

bluedark

|                                                                                              |                                                                                                           |
|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| tablerow Java 17 module system blocked AssertJ-Swing from accessing internal Java fields     | Added <code>-add-opens</code> JVM arguments to <code>maven-surefire-plugin</code> in <code>pom.xml</code> |
| white SonarCloud showed 0% coverage despite JaCoCo generating reports                        | Added <code>sonar.coverage.jacoco.xmlReportPaths</code> property pointing to the correct XML path         |
| tablerow SonarCloud Automatic Analysis conflicted with CI-based analysis                     | Disabled Automatic Analysis in SonarCloud Administration → Analysis Method                                |
| white <code>requireEnabled()</code> not recognised as assertion by SonarCloud                | Replaced all <code>require*()</code> calls with <code>assertThat(...).isTrue()/isFalse()</code>           |
| tablerow <code>MongoClient</code> resource leak flagged by SonarCloud (E Reliability rating) | Stored <code>MongoClient</code> as a <code>final</code> field instead of a local variable                 |
| white GUI tests fail on macOS with focus change errors                                       | Reordered text field interactions in tests to avoid focus issues                                          |
| tablerow E2E test comparison failed due to uppercase text conversion                         | Updated test values to match the actual display format                                                    |

## 7. Build and Test Instructions

---

The following instructions assume that **Java 17**, **Maven**, and **Docker** are already installed on the machine.

### 7.1 Clone the Repository

```
git clone https://github.com/mohamedjama-farah/product-catalog.git
cd product-catalog
```

### 7.2 Run the Full Build and All Tests

```
mvn clean verify
```

This single command compiles the code, runs all unit tests, integration tests, and end-to-end tests, and generates the JaCoCo coverage report. Docker must be running for Testcontainers to start the MongoDB container automatically.

### 7.3 Run Mutation Testing

```
mvn org.pitest:pitest-maven:mutationCoverage
```

### 7.4 Import into Eclipse

- **File** → **Import** → **Existing Maven Projects**
- Select the cloned repository folder
- Eclipse detects `pom.xml` and configures Java 17 automatically
- All Eclipse metadata files (`.project`, `.classpath`, `.settings`) are already included in the repository

### 7.5 Run Tests from Eclipse

All tests can be run from Eclipse using **Right-click** → **Run As** → **JUnit Test**. Docker must be running so that Testcontainers can start MongoDB automatically.

## 8. Summary

---

The Product Catalog project successfully implements all mandatory requirements of the Automated Software Testing course exam guidelines.

bluedark

|                                    |                                                       |
|------------------------------------|-------------------------------------------------------|
| tablerow Test-Driven Development   | All production code written test-first                |
| white Unit Tests                   | 38 unit tests (controller, model, view)               |
| tablerow Integration Tests         | 11 tests using real MongoDB via Testcontainers        |
| white End-to-End Tests             | 2 tests covering full application workflow            |
| tablerow Code Coverage (Coveralls) | 100%                                                  |
| white Code Coverage (SonarCloud)   | 100%                                                  |
| tablerow Mutation Testing (PIT)    | 8/8 mutations killed — 0 surviving mutants            |
| white SonarCloud Quality Gate      | PASSED — 0 issues, 0 duplications                     |
| tablerow No 3rd Party Mocking      | MongoClient never mocked — Testcontainers used        |
| white Build Automation             | Maven — single command: <code>mvn clean verify</code> |
| tablerow Continuous Integration    | GitHub Actions — Linux — all tests green              |
| white Java Version                 | Java 17 LTS                                           |
| tablerow GUI Framework             | Java Swing — tested with AssertJ-Swing                |
| white Database                     | MongoDB 6.0 via Testcontainers                        |